CIS 5511 – Homework 4

Md. Mizanur Rahman
md.mizanur.rahman@temple.edu

October 2, 2026

Task 1

Recursive Selection Sort

Algorithm

Selection Sort repeatedly selects the largest element in the current unsorted portion and places it at the end. A recursive version sorts a problem of size n by first placing the largest of the n elements at position $n-1$, and then recursively sorting the first $n-1$ elements.

```
RECURSIVE-SELECTION-SORT(A, n)

1. if n <= 1
2.     return
3. maxIndex = 0
4. for i = 1 to n - 1
5.     if A[i] > A[maxIndex]
6.         maxIndex = i
7. exchange A[maxIndex] with A[n - 1]
8. RECURSIVE-SELECTION-SORT(A, n - 1)
```

The initial call is

```
RECURSIVE-SELECTION-SORT(A, length(A))
```

Explanation

For example, suppose the array is

```
[5, 2, 4, 1, 3]
```

The first call finds that the maximum value is 5 and exchanges it with the final element:

```
[3, 2, 4, 1, 5]
```

Now 5 is in its correct final position. The algorithm recursively sorts only the first four elements. The largest value in that portion is 4, giving

```
[3, 2, 1, 4, 5]
```

The process continues until only one element remains.

Correctness

For a subproblem containing n elements, the linear scan finds the maximum element and places it at position $n-1$. Therefore, that element is in its final sorted position.

The recursive call correctly sorts the remaining first $n-1$ elements. Consequently, after the recursive call finishes, the entire array is sorted.

The base case is $n \leq 1$, where an array containing zero or one element is already sorted.

Time Analysis

For a problem of size n, the maximum-selection step performs

$$n-1$$

key comparisons.

The recursive problem has size $n-1$. Therefore,

$$T(n) = T(n-1) + \Theta(n).$$

Expanding the recurrence,

$$T(n) = (n-1) + (n-2) + \cdots + 2 + 1.$$

Using

$$1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2},$$

we obtain

$$T(n) = \Theta(n^2).$$

The number of comparisons does not depend on the initial arrangement of the data. Therefore,

$$\boxed{T_{\text{best}}(n) = T_{\text{average}}(n) = T_{\text{worst}}(n) = \Theta(n^2)}$$

for recursive Selection Sort.

Recursive Insertion Sort

Algorithm

Recursive Insertion Sort first recursively sorts the first $n-1$ elements and then inserts the last element into the proper position in the sorted prefix.

```
RECURSIVE-INSERTION-SORT(A, n)

1. if n <= 1
2.     return

3. RECURSIVE-INSERTION-SORT(A, n - 1)

4. key = A[n - 1]
```

```
5. j = n - 2

6. while j >= 0 and A[j] > key
7.     A[j + 1] = A[j]
8.     j = j - 1

9. A[j + 1] = key
```

The initial call is

```
RECURSIVE-INSERTION-SORT(A, length(A))
```

Explanation

Consider

```
[5, 2, 4, 1]
```

The recursion first reaches

```
[5]
```

which is already sorted.
The value 2 is inserted:

```
[2, 5]
```

Next, 4 is inserted:

```
[2, 4, 5]
```

Finally, 1 is inserted:

```
[1, 2, 4, 5]
```

Each recursive level therefore assumes that the prefix is already sorted and inserts one additional element.

Correctness

Assume recursively that the first $n - 1$ elements have been correctly sorted.

The insertion step stores the final value as `key` and shifts every preceding element larger than `key` one position to the right. When the loop stops, `key` is placed immediately after the last value not greater than it.

Therefore, the first n elements are sorted.

The base case is $n \leq 1$, which is already sorted.

Worst-Case Analysis

The worst case occurs when the array is in reverse order. When inserting the nth element, as many as $n-1$ previously sorted elements must be examined and shifted.

Thus,

$$T(n) = T(n-1) + \Theta(n).$$

Therefore,

$$\boxed{T_{\text{worst}}(n) = \Theta(n^2)}$$

Best-Case Analysis

The best case occurs when the input is already sorted. At each recursive level, the test

```
A[j] > key
```

fails immediately.

Therefore,

$$T(n) = T(n-1) + \Theta(1),$$

which gives

$$\boxed{T_{\text{best}}(n) = \Theta(n)}$$

Average-Case Analysis

For randomly ordered distinct elements, when a value is inserted into an already sorted prefix, on average a constant fraction of the prefix must be examined or shifted.

Thus,

$$T(n) = T(n-1) + \Theta(n),$$

and consequently,

$$\boxed{T_{\text{average}}(n) = \Theta(n^2)}$$

The two recursive sorting algorithms can therefore be summarized as

Case	Selection Sort	Insertion Sort
Best	$\Theta(n^2)$	$\Theta(n)$
Average	$\Theta(n^2)$	$\Theta(n^2)$
Worst	$\Theta(n^2)$	$\Theta(n^2)$

Insertion Sort can take advantage of already sorted or nearly sorted input, whereas Selection Sort performs approximately the same number of comparisons regardless of the original ordering.

Task 2

Lomuto Partition and Hoare Partition

Let the subarray being partitioned contain

$$n = r - p + 1$$

elements.
To make the comparison precise, I count two operations:

1. comparisons between an array value and the pivot;
2. exchanges between two array elements.

For the exact average-case results, I assume that all n values are distinct, all $n!$ relative orderings are equally likely, and one execution of an exchange statement counts as one exchange.

Lomuto Partition

The Lomuto algorithm used in the lecture takes the last element as the pivot.

```
LOMUTO-PARTITION(A, p, r)

1. x = A[r]
2. i = p - 1
3. for j = p to r - 1
4.     if A[j] <= x
5.         i = i + 1
6.         exchange A[i] with A[j]
7. exchange A[i + 1] with A[r]
8. return i + 1
```

For distinct elements, the algorithm places the pivot into its final position.

Number of Comparisons

The loop examines every value except the pivot exactly once. Therefore,

$$\boxed{C_L(n) = n - 1}$$

key-to-pivot comparisons are performed.
Thus,

$$C_L(n) = \Theta(n).$$

Number of Exchanges

Suppose the pivot is the kth smallest element.

Exactly $k-1$ elements are smaller than the pivot. For each such element, an exchange statement is executed inside the loop.

One additional exchange occurs after the loop to place the pivot.

Therefore,

$$S_L(k) = (k-1) + 1 = k.$$

Hence,

$$\boxed{S_L(k) = k}$$

The minimum occurs for $k = 1$:

$$\boxed{S_{L,\min} = 1}$$

and the maximum occurs for $k = n$:

$$\boxed{S_{L,\max} = n}$$

This count includes an exchange even when the two indices are equal, because the pseudocode still executes the exchange statement.

Expected Number of Exchanges

Since every pivot rank is equally likely,

$$P(K = k) = \frac{1}{n}.$$

Therefore,

$$E[S_L] = \frac{1}{n} \sum_{k=1}^{n} k.$$

Using

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2},$$

we obtain

$$\boxed{E[S_L] = \frac{n+1}{2}}$$

which is approximately $n/2$.

Hoare Partition

Hoare Partition uses two indices that move toward one another. The lecture version uses the first element as the pivot.

```
HOARE-PARTITION(A, p, r)

1. x = A[p]
2. i = p - 1
3. j = r + 1
4. while TRUE
5.     repeat
6.         j = j - 1
7.     until A[j] <= x
8.     repeat
9.         i = i + 1
10.    until A[i] >= x
11.    if i < j
12.        exchange A[i] with A[j]
13.    else
14.        return j
```

Unlike Lomuto Partition, Hoare Partition does not necessarily place the pivot in its final sorted position. Instead, it returns a partition boundary.

Number of Comparisons

For distinct values, the two scanning loops together perform either

$$n+1$$

or

$$n+2$$

key-to-pivot comparisons.
If the pivot is the smallest element, the count is $n+1$. Otherwise, the count is $n+2$.
A random first element is the smallest with probability $1/n$. Therefore,

$$E[C_H] = \frac{1}{n}(n+1) + \frac{n-1}{n}(n+2).$$

Simplifying,

$$\boxed{E[C_H] = n + 2 - \frac{1}{n}}$$

Thus Hoare performs slightly more comparisons than Lomuto, but both are linear:

$$\boxed{C_L(n) = \Theta(n), \qquad C_H(n) = \Theta(n)}$$

Number of Exchanges

Each successful Hoare exchange corrects two misplaced elements at once.

The minimum number of exchanges is

$$\boxed{0},$$

while the maximum is

$$\boxed{\left\lfloor \frac{n}{2} \right\rfloor}.$$

Therefore,

$$0 \leq S_H(n) \leq \left\lfloor \frac{n}{2} \right\rfloor.$$

Expected Number of Exchanges

Suppose the pivot has rank k.

There are $k-1$ elements smaller than the pivot and $n-k$ elements larger than the pivot.

For a fixed pivot rank k, the expected number of exchanges is

$$E[S_H \mid K = k] = \frac{(k-1)(n-k+1)}{n-1}.$$

Averaging over all pivot ranks gives

$$E[S_H] = \frac{1}{n} \sum_{k=1}^{n} \frac{(k-1)(n-k+1)}{n-1}.$$

Let

$$t = k - 1.$$

Then

$$E[S_H] = \frac{1}{n(n-1)} \sum_{t=0}^{n-1} t(n-t).$$

Expanding the sum,

$$\sum_{t=0}^{n-1} t(n-t) = n \sum_{t=0}^{n-1} t - \sum_{t=0}^{n-1} t^2.$$

Using

$$\sum_{t=0}^{n-1} t = \frac{n(n-1)}{2}$$

and

$$\sum_{t=0}^{n-1} t^2 = \frac{(n-1)n(2n-1)}{6},$$

we obtain

$$\sum_{t=0}^{n-1} t(n-t) = \frac{n(n-1)(n+1)}{6}.$$

Therefore,

$$\boxed{E[S_H] = \frac{n+1}{6}}$$

which is approximately $n/6$.

Direct Comparison

The results are summarized below.

Property	Lomuto	Hoare
Pivot in lecture version	Last element	First element
Final pivot position returned	Yes	Not necessarily
Key comparisons	$n-1$	$n+1$ or $n+2$
Expected comparisons	$n-1$	$n+2-\frac{1}{n}$
Minimum exchanges	1	0
Maximum exchanges	n	$\lfloor n/2 \rfloor$
Expected exchanges	$\frac{n+1}{2}$	$\frac{n+1}{6}$
Partition time	$\Theta(n)$	$\Theta(n)$
Extra array space	$\Theta(1)$	$\Theta(1)$

Both partition algorithms therefore have

$$\boxed{T_L(n) = T_H(n) = \Theta(n)}$$

running time for a single partition operation.
However, their constant-factor behavior differs.
Lomuto performs approximately

$$\frac{n}{2}$$

exchanges on average, whereas Hoare performs approximately

$$\frac{n}{6}.$$

Therefore,

$$\frac{E[S_L]}{E[S_H]} = \frac{(n+1)/2}{(n+1)/6} = 3.$$

Thus Lomuto performs approximately three times as many exchange statements as Hoare on a random permutation of distinct values.

An exchange normally requires several assignments:

```
temp = A[i]
A[i] = A[j]
A[j] = temp
```

Therefore reducing the number of exchanges can significantly reduce the amount of data movement.

Lomuto performs slightly fewer comparisons, while Hoare performs substantially fewer exchanges.

Behavior with Duplicate Values

Duplicate values also reveal an important difference.

Suppose all values are identical:

```
[5, 5, 5, 5, 5]
```

For Lomuto, the condition

```
A[j] <= pivot
```

is true for every scanned value.

Therefore an exchange is executed for each scanned element, followed by the final pivot exchange. The pivot is also placed at the extreme end of the partition.

If such behavior repeatedly occurs in Quicksort, the recurrence can become

$$T(n) = T(n-1) + \Theta(n),$$

which gives

$$\Theta(n^2).$$

Hoare behaves differently. Its two pointers stop on equal values from opposite directions and continue moving inward after exchanges. Therefore equal values tend to be distributed across the two resulting regions rather than always creating an extremely unbalanced partition.

Hence, Hoare Partition is generally more favorable when many duplicate values occur.

Summary of Lomuto and Hoare Partition

Both Lomuto Partition and Hoare Partition have linear running time, $\Theta(n)$, because each processes the elements of the subarray only a constant number of times. However, asymptotic analysis alone does not show their practical difference.

For a subarray of n distinct elements, Lomuto performs exactly $n-1$ element-to-pivot comparisons and between 1 and n exchanges. Under the assumption that every permutation is equally likely, its expected number of exchanges is $(n+1)/2$.

Hoare performs approximately $n+1$ to $n+2$ element-to-pivot comparisons, but only between 0 and $\lfloor n/2 \rfloor$ exchanges, with an expected number of approximately $(n+1)/6$.

Therefore, Lomuto performs slightly fewer comparisons, while Hoare performs about three times fewer exchanges on random distinct input. Since exchanging two elements requires several data movements, Hoare is generally more efficient with respect to data movement.

Hoare also handles large numbers of equal values more favorably because its two pointers tend to divide equal elements between the two partitions, whereas Lomuto's `<=` rule may create a highly unbalanced partition. Nevertheless, both partition algorithms remain $\Theta(n)$ asymptotically.

Conclusion

Recursive Selection Sort and recursive Insertion Sort both reduce a problem of size n to one recursive problem of size $n-1$.

Selection Sort performs linear preprocessing before recursion:

$$\text{select maximum} \rightarrow \text{recursive sort.}$$

Its best, average, and worst running times are all

$$\Theta(n^2).$$

Insertion Sort performs its insertion after recursively sorting the prefix:

$$\text{recursive sort} \rightarrow \text{insert last element.}$$

Its best-case running time is

$$\Theta(n),$$

while its average and worst cases are

$$\Theta(n^2).$$

Both Lomuto Partition and Hoare Partition require

$$\Theta(n)$$

time and

$$\Theta(1)$$

extra space.

However, a more accurate operation count shows an important difference. Lomuto performs exactly $n-1$ key-to-pivot comparisons and an expected

$$\frac{n+1}{2}$$

exchanges.

Hoare performs approximately $n+2$ key-to-pivot comparisons but only an expected

$$\frac{n+1}{6}$$

exchanges.

Therefore, Lomuto uses slightly fewer comparisons, while Hoare performs approximately three times fewer exchanges for random distinct input. Since an exchange requires multiple data assignments, Hoare can have lower practical data-movement cost even though both algorithms have the same asymptotic partition complexity.

AI use Acknowledgment

Generative AI (ChatGPT) was used to assist with LaTeX formatting, some part of writing and presentation of this report.